

Разработка децентрализованных приложений

Децентрализованных приложений

Децентрализация

Концепция децентрализации не нова. Она давно используется в стратегическом планировании, управлении и правительствах. Основная идея децентрализации заключается в распределении контроля и полномочий между разными отделами организации, вместо того чтобы управлять всем централизованно. Подобная конфигурация дает несколько преимуществ, таких как повышение эффективности, ускоренное принятие решений, улучшение мотивации и снижение нагрузки на высшее руководство.

Блокчейн основывается на отсутствии централизованного контроля; мы представим примеры различных способов децентрализации и того, как их можно достичь. Мы также подробно обсудим децентрализацию экосистемы блокчейна, распределенные приложения и платформы для обеспечения децентрализации. Кроме того, вам будет продемонстрировано несколько потрясающих программ и идей, основанных на технологии децентрализованного блокчейна.

Децентрализация с помощью блокчейна

Децентрализация является основным преимуществом, предоставляемым технологией блокчейн, ее ключевым сервисом. Блокчейн задумывался как идеальный транспортный механизм для создания платформ, которые не нуждаются в промежуточных звеньях и могут функционировать со множеством разных ведущих узлов, выбираемых через механизм консенсуса. Эта модель делает возможной неограниченную конкуренцию за право принимать решения. Такая конкуренция управляется механизмом консенсуса, и наиболее распространенный метод, который при этом используется, известен под названием PoW (англ. Proof of Work – доказательство выполнения работы).

Модель децентрализации может быть как полной, так и частичной, в зависимости от требований и обстоятельств. С точки зрения блокчейна децентрализацию можно рассматривать в качестве механизма, позволяющего создавать новые или преобразовывать уже имеющиеся приложения и парадигмы с целью предоставления пользователям полного контроля.

Информационно-коммуникационные технологии (ИКТ) традиционно основываются на централизованной парадигме, в которой база данных или серверы приложений находятся под чьим-то единоличным управлением – напри мер, системного администратора. С возникновением биткойна и технологии блокчейн эта модель претерпела ряд изменений, и теперь существуют техно логии, которые позволяют кому угодно создать децентрализованную систему и управлять ею без единой точки отказа или единого доверенного лица. Она может работать автономно или требовать определенного человеческого вмешательства – это зависит от типа и модели управления, которые используются в децентрализованных приложениях, выполняющихся в рамках блокчейна. На рис. 20 представлены разные варианты существующих систем: центральные, децентрализованные и распределенные. Впервые эта концепция была предложена Полом Бэраном в книге «On Distributed Communications: I. Introduction to Distributed Communications Networks» (Rand Corporation, 1964).

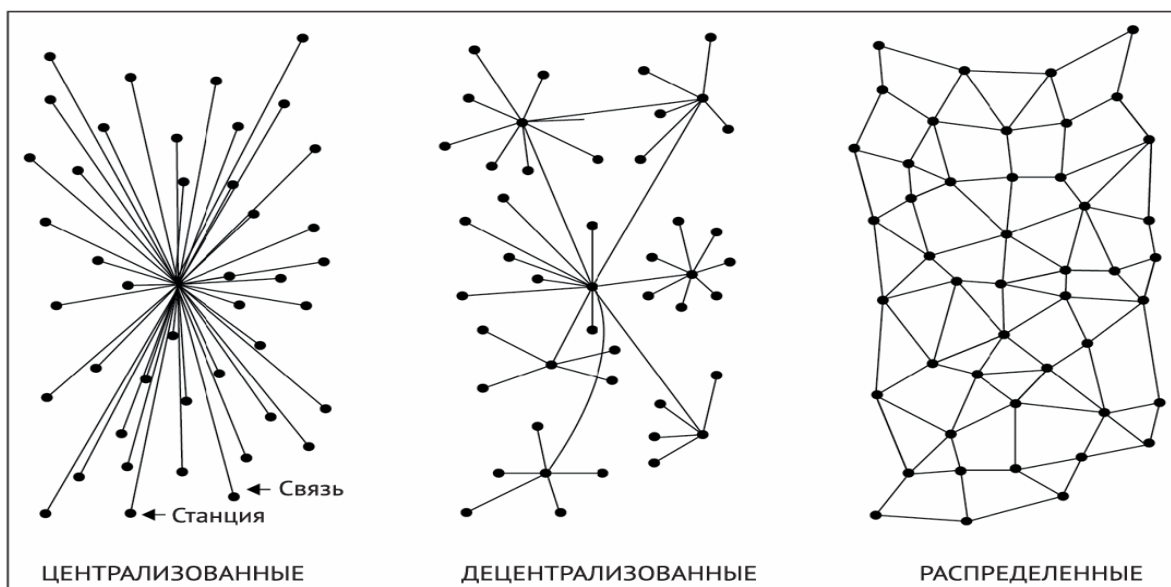


Рис 20 - Разные виды сетей/систем

Централизованные (клиент-серверные) системы являются традиционными в мире информационных технологий. Управление в них осуществляется из единой точки, которая полностью отвечает за работу системы. Все пользователи получают услуги из единого источника. Большинство поставщиков интернет сервисов, включая Google, Amazon, eBay, App Store от компании Apple и многие другие, используют эту устоявшуюся модель для предоставления услуг.

В распределенных системах данные и вычисления «размазаны» по не скольким узлам в сети. Иногда этот термин путают с параллельными вычислениями. Определения этих концепций пересекаются между собой, однако ключевое отличие состоит в том, что для достижения результата параллельные вычисления выполняются одновременно всеми узлами; такой подход, к примеру, используется в анализе и прогнозе погоды, симуляциях и финансовом моделировании. Для сравнения: в распределенных системах вычисления могут происходить не параллельно, а данные копируются на несколько узлов, которые воспринимаются пользователями как единая, слаженная система. Разновидности обеих этих моделей используются для достижения отказоустойчивости и высокой скорости. В системах с параллелизацией по-прежнему присутствует центральный механизм управления, который контролирует все узлы, отвечающие за обработку данных. Это означает, что такие системы по своей сути являются централизованными.

Ключевое отличие между децентрализованными и распределенными системами заключается в том, что последние по-прежнему содержат центральный механизм управления, который полностью их контролирует; в децентрализованных системах такого механизма нет.

Децентрализованная система является разновидностью сети, отдельные части которой не зависят от единого главного узла; вместо этого управление распределяется между многими узлами. В качестве аналогии можно привести организацию, в которой каждый отдел отвечает за собственный сервер баз данных; таким образом полномочия снимаются с центрального сервера и распределяются между разными отделами. Важной инновацией в этой парадигме, которая дала начало эпохе децентрализованных приложений, является децентрализованный консенсус. Этот механизм возник благодаря биткойну. С помощью алгоритма консенсуса пользователи могут принимать решения без использования централизованных, доверенных решений, таких как сторонние механизмы, посредники или поставщики услуг.

Методы Децентрализации

Децентрализации можно добиться двумя способами: за счет избавления от посредников или введения конкуренции (децентрализация, основанная на состязании). Эти методы подробно обсуждаются в следующих разделах.

Избавление от посредников

Концепцию избавления от посредников можно объяснить на следующем примере. Представьте, что вы хотите отправить деньги за границу своему другу. Вы обращаетесь в банк, который бесплатно передаст ваши деньги другому банку в нужной вам стране. В этом случае банк использует центральную базу данных, которая фактом своего обновления подтверждает отправку денежного перевода. Технология блокчейн позволяет отправить деньги непосредственно вашему другу, без использования услуг банка. Все, что вам нужно, – это адрес вашего друга в блокчейне. Таким образом отпадает необходимость в посреднике (то есть банке) и достигается децентрализация. Практичность подобного подхода в финансовом секторе, в котором присутствует огромное количество регуляций и требований к соблюдению различных норм, остается под вопросом. Тем не менее, помимо финансов, данная модель может использоваться во многих других отраслях.

Децентрализация на основе состязания

В подходе с использованием конкуренции разные поставщики услуг конкурируют между собой; победитель, выбранный системой, будет предоставлять услуги. И хотя эта методика не обеспечивает полной децентрализации, она в определенной степени гарантирует, что ни один посредник или поставщик услуг не станет монополистом. В контексте технологии блокчейн можно спроектировать такую систему, в которой смарт-контракты могут выбирать внешние источники данных из большого количества претендентов в зависимости от их репутации, предыдущей оценки, отзывов и качества обслуживания.

Такой подход не приводит к полной децентрализации, однако он позволяет смарт-контрактам делать свободный выбор на основе вышеупомянутых критериев. Таким образом среди поставщиков услуг развивается конкурентная среда, в которой они состязаются друг с другом за право быть избранным.

На рис. 21 представлены различные уровни децентрализации. Слева показан традиционный подход, при котором контроль принадлежит центральной системе; в правой части можно наблюдать полное избавление от посредников. В центре размещен метод с конкурирующими посредниками или поставщиками услуг, которые выбираются на основе репутации или голосования. На этом уровне достигается частичная децентрализация.



Рис 21 - Шкала децентрализации

Несмотря на множество преимуществ, таких как прозрачность, эффективность, экономия средств, разработка доверенных экосистем и, в некоторых случаях, конфиденциальность и анонимность, некоторые аспекты децентрализации, включая требования к безопасности, программные ошибки и человеческий фактор, нуждаются в пристальном рассмотрении.

Например, в децентрализованных системах, таких как биткойн или Ethereum, безопасность обычно обеспечивается за счет частных ключей. Как в этом случае гарантировать, что при потере этих ключей (в результате ошибки в коде смарт-контракта или уязвимости в децентрализованном приложении) связанная с ними смарт-собственность не станет бесполезной? Прежде чем приступать к децентрализации всего и вся с помощью блокчейна и децентрализованных приложений, необходимо понимать, что не все на свете может или должно быть децентрализованным.

В связи с этим встает несколько основополагающих вопросов. Действительно ли нам нужна технология блокчейн? Если да, то когда? И в каких ситуациях она более предпочтительная, по сравнению с традиционными базами данных? Чтобы в этом разобраться, давайте пройдемся по простому списку с вопроса ми, представленному ниже:

1. Требуется ли высокая пропускная способность данных? Если ответ положительный, используйте традиционную базу данных.
2. Контролируются ли обновления централизованно? Если да, используйте традиционную базу данных.
3. Доверяют ли пользователи друг другу? Если да, используйте традиционную базу данных.
4. Являются ли пользователи анонимными? Если да, используйте публичный блокчейн, если нет – частный.
5. Если в рамках консорциума необходимо обеспечивать консенсус, используйте частный блокчейн, если нет – публичный.

Ответив на все эти вопросы, можно понять, есть ли необходимость в техно логии блокчейн. Но, помимо вопросов, поднятых в данной модели, существует множество других аспектов, которые следует учитывать, например задержка сети, выбор механизма консенсуса, нужен ли консенсус вообще, и если да, то где он будет достигнут. Если консенсус обеспечивается внутри за счет консорциума, следует использовать частный блокчейн; публичный блокчейн стоит рассматривать, если консенсус должен достигаться открыто между несколькими сущностями. При выборе между блокчейном и традиционными базами данных необходимо учитывать и другие аспекты, такие как неизменяемость. Если требуется строгая неизменяемость данных, следует выбрать публичный блокчейн; в противном случае может подойти центральная база данных.

По мере того как технология блокчейн становится все более зрелой, к ней будут предъявляться дополнительные требования. Но пока что данного набора вопросов достаточно, что понять, стоит ли основывать свои решения на ней или нет.

Пути децентрализации

Блокчейну и биткойну предшествовали другие технологии, такие как BitTorrent и файловая система Gnutella, которые в определенной степени можно считать децентрализованными. Однако именно появление блокчейна дало толчок множеству инициатив по достижению децентрализации. Для многих первой в списке на рассмотрение является технология биткойн, которая зарекомендовала себя как наиболее устойчивая и безопасная разновидность блокчейна; кроме того, на момент написания этих строк ее рыночная капитализация составляет 145 миллиардов долларов. Многие разработчики используют для построения децентрализованных приложений альтернативные версии блок чейна.

Например, на фоне биткойна начинает выделяться Ethereum; благодаря своей гибкости эта система позволяет внедрять в блокчейн любую бизнес-логику, используя смарт-контракты.

Как происходит децентрализация

В своей книге «Bitcoin and Cryptocurrency Technologies» (Princeton University Press) Арвинд Нараянан вместе с соавторами предлагает критерии, с помощью которых можно оценивать требования к децентрализации различных аспектов системы в контексте технологии блокчейн. Эти критерии состоят из четырех вопросов, ответы на которые дают четкое понимание того, как именно можно децентрализовать систему:

1. Что подлежит децентрализации?
2. Какая степень децентрализации требуется?
3. Какая разновидность блокчейна используется?
4. Какой механизм безопасности используется?

Первый вопрос всего лишь предлагает вам определиться с тем, что именно децентрализуется. Это может быть система идентификации или торговая платформа.

При ответе на второй вопрос нужно указать требуемую степень децентрализации путем анализа масштабов этого процесса, как упоминалось ранее. Это может быть как полное, так и частичное избавление от посредников.

В третьем пункте разработчиков просят определить, какая разновидность блокчейна подходит для конкретного приложения. Это может быть биткойн, Ethereum или любая другая сеть, отвечающая требованиям определенного проекта.

Наконец, необходимо ответить на фундаментальный вопрос: каким образом будет обеспечена безопасность децентрализованной системы? Механизм безопасности, к примеру, может быть основан на атомарности, когда каждая транзакция либо выполняется полностью, либо не выполняется вообще. Такой детерминистический подход гарантирует целостность системы. В числе других можно применять механизм, основанный на репутации, который позволяет достигать разной степени доверия в системе.

Пример использования критериев децентрализации

Давайте в качестве примера приложения, выбранного для децентрализации, возьмем систему денежных переводов. Для оценки требований к децентрализации этой системы ответим на четыре вопроса, которые мы обсуждали выше:

- 1) система денежных переводов;
- 2) избавление от посредников;
- 3) биткойн;
- 4) атомарность.

Ответы говорят о том, что система денежных переводов может быть децентрализована путем избавления от промежуточных звеньев и реализована на основе сети биткойн, а гарантии безопасности будут обеспечены за счет атомарности. Благодаря атомарности транзакции выполняются либо успешно и в полном объеме, либо никак. Мы выбрали биткойн, потому что это наиболее устоявшаяся технология блокчейн, которая прошла проверку временем.

Данные критерии можно использовать аналогичным образом в любой другой системе, которую необходимо оценить с точки зрения децентрализации. Ответы на эти четыре простых вопроса помогут определиться с подходом к децентрализации системы.

Блокчейн и полная децентрализация экосистемы

Для достижения полной децентрализации необходимо, чтобы среда вокруг блокчейна тоже была децентрализованной. Блокчейн представляет собой распределенный журнал, работающий поверх традиционных элементов, включая хранилище данных, коммуникационный протокол и вычисления. Существуют и другие факторы, которые традиционно основываются на централизованных парадигмах (например, подлинность и благосостояние); для получения достаточно децентрализованной экосистемы их децентрализация не требуется.

Хранилище данных

Данные могут храниться непосредственно в блокчейне – этим и достигается их децентрализация. Однако существенным недостатком такого подхода является то, что блокчейн по своей природе не подходит для хранения больших объемов информации. В нем могут накапливаться простые транзакции и произвольные данные, но, в отличие от традиционных баз данных, эта технология совершенно не подходит для хранения изображений или больших бинарных файлов.

Более предпочтительной альтернативой для хранения данных является использование распределенных хеш-таблиц (англ. Distributed Hash Tables, или DHT). DHT изначально применялись для обмена файлами в одноранговых сетях, таких как BitTorrent, Napster, Kazaa и Gnutella. Их исследование стало популярным благодаря таким проектам, как CAN, Chord, Pastry и Tapestry. BitTorrent является самой масштабируемой и быстрой сетью, но ей и ее аналогам свойственна одна проблема: у пользователей нет стимула хранить данные бесконечно. Обычно файлы хранятся ограниченное время, и если узлы, на которых они находятся, отключатся от сети, их невозможно будет получить – придется ждать, когда соответствующие узлы опять подключатся и файлы снова станут доступными.

Двумя основными требованиями являются высокая доступность и стабильность узлов; это означает, что данные должны быть доступны по запросу и что к звеньям сети всегда можно обратиться. Файловая система IPFS, разработанная Хуаном Бенетом, обладает обоими этими качествами; ее виденье заключается в предоставлении децентрализованной «всемирной паутины» путем замены протокола HTTP. Для хранения данных и поисковых функций IPFS использует Kademlia DHT и, соответственно, направленный ациклический граф Меркла (англ. Directed Acyclic Graph или DAG). Подробно с концепциями DHT и DAG вы познакомитесь в главе 4. Стимул для хранения данных основан на протоколе под названием Filecoin, который вознаграждает узлы, хранящие данные, с использованием механизма Bitswap. Последний дает возможность узлам вести простой журнал отправленных и полученных байтов в отношении «один-к-одному». Кроме того, в IPFS применяется система контроля версий на основе Git, которая помогает структурировать данные и управлять их версионированием.

Есть и другие решения для хранения данных, такие как Ethereum Swarm, Storj и MaidSafe. У Ethereum есть собственная децентрализованная и распределенная экосистема, которая использует Swarm в качестве хранилища и протокол Whisper для коммуникаций. MaidSafe стремится построить децентрализованный интернет. Все эти проекты подробнейшим образом описаны далее в этой книге. Еще одним проектом для децентрализации уровня хранения информации является BigchainDB, который нацелен на предоставление линейно масштабируемой и быстрой децентрализованной базы данных в противовес традиционным

файловым системам. BigchainDB дополняет децентрализованные вычислительные платформы и файловые системы, такие как Ethereum и IPFS.

Вычислительная мощность и децентрализация

Децентрализация вычислительных мощностей достигается за счет таких блокчейновых технологий, как Ethereum, которые выполняют смарт-контракты со встроенной бизнес-логикой. Другие разновидности блокчейна предоставляют похожие вычислительные платформы, в которых бизнес-логика пропускается через сеть децентрализованным способом.

На рис. 22 показана упрощенная схема децентрализованной экосистемы. Нижний уровень (интернет или меш-сети) предоставляет слой децентрализованных коммуникаций. Чуть выше находится слой хранения данных, в котором децентрализация достигается за счет таких технологий, как IPFS и BigchainDB. На следующем уровне вы можете видеть сам блокчейн, который выступает в роли слоя децентрализованных вычислений. Блокчейн также предоставляет ограниченные средства хранения данных, но это существенно снижает скорость и пропускную способность системы. По этой причине для хранения больших объемов информации децентрализованным способом лучше использовать другие решения, такие как IPFS и BigchainDB. В самом верху находятся слои подлинности и благосостояния. Подлинность в Интернете – это обширная тема; такие системы, как BitAuth и OpenID, предоставляют услуги аутентификации и идентификации различной степени децентрализованности и безопасности.



Рис 22 - Децентрализованная экосистем

Технология блокчейн способна решить различные проблемы, связанные с децентрализацией. Концепция под названием «треугольник Зуко», относящаяся к подлинности, требует, чтобы система именования в сетевом протоколе была безопасной, децентрализованной и способной предоставить пользователям понятные и запоминающиеся имена. Предполагается, что подобная система может обладать одновременно лишь двумя из этих свойств. Тем не менее с появлением такой разновидности блокчейна, как Namescoin, данная проблема была решена. Благодаря Namescoin мы можем добиться безопасности, децентрализованности и понятных для человека имен. Но такой подход не является панацеей и имеет свои особенности – например, пользователи сами должны отвечать за безопасное хранение своих приватных ключей. Это ставит новые вопросы общего характера относительно того, подходит ли децентрализация для решения той или иной проблемы.

Децентрализация уместна не во всех случаях. Централизованные системы с прочно устоявшейся репутацией зачастую работают лучше. Например, системы электронной почты от известных компаний, таких как Google или Microsoft, предоставляют более качественные

услуги, по сравнению с отдельными почтовыми серверами, принадлежащими интернет-пользователям.

В настоящее время в процессе разработки находится множество проектов, нацеленных на создание более комплексных распределенных блокчейнов. На пример, для децентрализованного хранения данных и коммуникации в блок чейне Ethereum были созданы системы Swarm и Whisper. Swarm и Ethereum.

С возникновением концепции децентрализации в прессе и академической литературе начали использовать разную терминологию и модные словечки. Благодаря изобретению технологии блокчейн стала возможной реализация традиционных физических структур в виде так называемых децентрализованных организаций (ДО) и аналогичных конструкций, которые мы по дробно рассмотрим чуть ниже. В контексте децентрализации следующие концепции заслуживают отдельного внимания.

Смарт-контракты

Смарт-контракты – это децентрализованные программы. Они могут работать автономно, однако ввиду преимуществ, связанных с безопасностью, блокчейн является стандартной децентрализованной платформой для их выполнения. Смарт-контракт обычно содержит некую бизнес-логику и данные ограниченного размера. Бизнес-логика выполняется в случае соответствия определенным критериям. Смарт-контракты используются акторами или участниками блокчейна или же выполняются автономно от их имени.

Децентрализованные Организации

Децентрализованные организации (ДО) представляют собой компьютерные программы, которые выполняются в блокчейне и являются отражением реальной организации с людьми и протоколами. При добавлении ДО в блокчейн в виде одного или нескольких смарт-контрактов она становится децентрализованной, и все участники процесса взаимодействуют между собой согласно логике, содержащейся в коде ДО.

Децентрализованные автономные организации

Как и ДО, децентрализованные автономные организации (ДАО) являются компьютерными программами, которые работают поверх блокчейна вместе со встроенными в них средствами управления и правилами бизнес-логики. ДАО и ДО принципиально ничем не отличаются. Разница лишь в том, что ДАО являются автономными; это означает, что они полностью автоматизированы и содержат логику на основе искусственного интеллекта. Этого нельзя сказать о ДО, которым для выполнения бизнес-логики нужен пользовательский ввод.

Возникновением первых ДАО мы обязаны сети Ethereum. В ДАО управляющей сущностью выступает код, а не люди или бумажные контракты. Однако поддержкой кода и оценкой предложений, исходящих от сообщества, занимается живой куратор. При получении достаточного количества входящих данных со стороны держателей токенов (участников) ДАО могут нанимать внешних подрядчиков.

Самая известная децентрализованная автономная организация называется ДАО; на этапе краудфандинга она собрала 168 миллионов долларов. Проект ДАО задумывался как фонд венчурного капитала, который должен предоставлять децентрализованную бизнес-модель, где нет единого владельца. К сожалению, из-за ошибки в коде проект ДАО был взломан, и миллионы долларов в виде валюты Ether (ETH) перетекли в дочернюю ДАО, созданную хакерами. Чтобы аннулировать последствия атаки и начать восстановление денежных средств в блокчейне Ethereum, необходимо было произвести жесткое ответвление (форк). Этот инцидент послужил началом дискуссии о безопасности, качестве и необходимости

в тщательном тестировании кода смарт-контрактов, чтобы обеспечить их неприкосновенность и сделать их достаточно подконтрольными. В процессе разработки находятся другие проекты (особенно в академической среде), призванные формализовать написание и тестирование смарт-контрактов.

В настоящее время ДАО не имеют никакого юридического статуса, хотя со держащийся в них код способен обеспечивать определенные протоколы и условия. Тем не менее пока что эти правила не имеют силы в реальной правовой системе. Возможно, когда-нибудь автономный агент (АА – участок кода, выполняющийся без человеческого вмешательства), введенный в эксплуатацию правоохранительными органами или государственным регулятором, будет со держать правила и регуляции, которые можно внедрить в ДАО с целью соблюдения ею правовых норм. Тот факт, что ДАО являются полностью децентрализованными, позволяет применять их в любой юрисдикции. Таким образом, они поднимают важный вопрос относительно того, как современная правовая система может функционировать в настолько разнообразном сочетании юридических и географических зон.

Децентрализованные автономные корпорации

Децентрализованные автономные корпорации (ДАК) по своему принципу похожи на ДАО, но считаются их подмножеством. Определения ДАК и ДАО иногда могут пересекаться, но основное их отличие состоит в том, что ДАО обычно являются некоммерческими, тогда как ДАК могут генерировать при быль путем начисления дивидендов держателям акций (участникам). ДАК может управлять бизнесом автоматически, без человеческого вмешательства, используя логику, которая в них запрограммирована.

Децентрализованные автономные общества

Децентрализованные автономные общества – это концепция, согласно ко торой в блокчейне может функционировать целое общество, используя множество сложных смарт-контрактов и сочетание ДАО и децентрализованных приложений, которые работают автономно. Эта модель не основана на либертарианской идеологии и не подразумевает бесплатность всего и вся. Она лишь помогает предоставлять посредством блокчейна многочисленные правительственные услуги, такие как система удостоверений личности, техническая инвентаризация и свидетельства о рождении. Согласно другой теории, если правительство коррумпировано, а централизованные институты власти не обеспечивают надлежащего уровня доверия, общество может основать в блок чейне свою собственную альтернативу, основанную на децентрализованном консенсусе и прозрачности. Эта идея выглядит как мечта либертарианца или нечто родом из киберпанка, однако в блокчейне она вполне реализуема.

Децентрализованные приложения

Все идеи, упомянутые выше, можно объединить в концепцию децентрализованных приложений (ДП). ДАО, ДАК и ДО представляют собой ДП, работающие поверх блокчейна в рамках одноранговой (англ. peer-to-peer, или P2P) сети. Они являют собой последние достижения в области децентрализации. С другой стороны, ДП являются компьютерными программами, которые могут выполняться в специальных или уже существующих блокчейнах, или просто использовать их протоколы. Такие приложения соответствуют трем категориям: тип I, тип II и тип III.

Требования к децентрализованным приложениям

Приложение считается децентрализованным, если оно отвечает следующим критериям. Это определение взято из первичного документа под названием «Общая теория децентрализованных приложений, ДП» авторства Дэвида Джонстона и др.

- ДП должно быть полностью автономным и иметь открытый исходный код; большинство его токенов не должно быть подконтрольно какой-то одной сущности. Любые изменения, вносимые в приложение, должны основываться на консенсусе и обратной связи, предоставляемой сообществом.
- Данные и записи всех операций приложения должны быть защищены с помощью криптографии и храниться в публичном, децентрализованном блокчейне, чтобы исключить наличие единой точки отказа.
- Приложение должно использовать криптографические токены для предоставления доступа и вознаграждения тем, кто приносит пользу, на пример майнерам биткойна.
- Токены должны быть сгенерированы ДП в соответствии со стандартным алгоритмом шифрования. Генерация токенов подтверждает их ценность для участников сети (например, майнеров).

Операции, проводимые децентрализованными приложениями

Достижение консенсуса в ДП может осуществляться с помощью таких алгоритмов, как PoW и PoS. Пока что только первый проявил себя чрезвычайно устойчивым к так называемым атакам 51-го процента (в чем можно убедиться на примере биткойна). Более того, ДП способны распределять токены (монеты) посредством майнинга, разработки и кампаний по сбору средств.

Примеры ДП

Ниже представлены некоторые примеры децентрализованных приложений.

KYC-Chain

Это приложение, основанное на смарт-контрактах, предоставляет безопасные и удобные средства управления данными, которые соответствуют требованиям KYC (Know Your Customer – знай своего клиента).

OpenBazaar

Это децентрализованная P2P-сеть, обеспечивающая прямые коммерческие отношения между продавцом и покупателем без необходимости в централизованном посреднике, таком как eBay или Amazon. Следует отметить, что в основе данной системы нет блокчейна; вместо этого в P2P-сети применяются хеш-таблицы, которые позволяют узлам общаться и обмениваться данными напрямую. В качестве платежного механизма это приложение использует Bitcoin и другие виды криптовалюты.

Lazooz

Это децентрализованный эквивалент Uber, который позволяет напрямую договариваться о совместных поездках. В качестве поощрения для пользователей используется доказательство передвижения, а также возможность заработать монеты Zooz.

Платформы для децентрализации

На сегодня доступно множество платформ для децентрализации. На самом деле предоставление децентрализации является фундаментальным свойством сетей блокчейн. Поэтому данную услугу можно предоставлять с помощью таких систем, как Bitcoin, Ethereum, Hyperledger Fabric или Quorum. Многие организации по всему миру предлагают

платформы, которые должны упрощать разработку распределенных приложений, делать их доступными и безопасными. Ниже описаны некоторые из этих платформ.

Ethereum

Начинает список Ethereum – первая разновидность блокчейна, предоставляющая тьюринг-полный язык и концепцию виртуальной машины. Она резко выделяется на фоне биткойна и других криптовалют с их ограниченными скриптовыми возможностями. Наличие тьюринг-полного языка под названием Solidity открывает перед разработчиками децентрализованных приложений безграничные возможности. Впервые этот публичный блокчейн, предназначенный для создания смарт-контрактов и децентрализованных приложений, был предложен в 2013 году Виталиком Бутериным. Валютные токены в Ethereum называются Ether (эфир).

MaidSafe

MaidSafe предоставляет сеть типа SAFE (англ. Secure Access For Everyone – безопасный доступ для всех), построенную на основе незадействованных ресурсов ее пользователей, таких как накопители, вычислительная мощь и соединения для передачи данных. Файлы в сети делятся на небольшие порции данных, которые шифруются и распространяются по узлам случайным образом. Эти данные могут быть получены лишь их владельцем. Одна из ключевых инноваций MaidSafe состоит в том, что сеть автоматически отвергает дубликаты файлов, что помогает снизить нагрузку и, соответственно, необходимость в дополнительных ресурсах. В качестве токена для поощрения участников используется Safecoin.

Lisk

Lisk – это платформа для создания блокчейн-приложений и криптовалют. Она позволяет разработчикам использовать JavaScript для построения децентрализованных приложений и размещать их в подходящих участках блокчейна. Для консенсуса Lisk использует механизм DPOS (англ. Delegated Proof of Stake – доказательство доли владения), при котором для защиты сети и предложения блоков можно выбрать 101 узел. В качестве серверной части выступают Node.js и JavaScript, тогда как на клиентской стороне можно использовать стандартные технологии, такие как CSS3, HTML5 и JavaScript. В виде валюты в блокчейне Lisk использует монету LSK. Еще одной производной Lisk является Rise – платформа для децентрализованных приложений и цифровой валюты, которая делает упор на безопасность системы. Более практическое знакомство с этими и другими платформами будет предложено в следующих главах.

Сеть Bitcoin

Сеть Bitcoin является пиринговой сетью (peer-to-peer), в которой узлы обмениваются информацией о транзакциях и блоках. В сети существуют разные типы узлов. Основных узлов два – это полные узлы и узлы SPV. Полные узлы, как понятно из названия, являются реализациями основных клиентов Bitcoin, которые несут функционал кошельков, майнинга, полного хранилища блокчейна и сетевой маршрутизации. Тем не менее такому узлу необязательно выполнять все из этих функций. Узлы SPV (Simple Payment Verification – простая верификация платежей) являются легковесными клиентами, которые выполняют только функции кошельков и сетевой маршрутизации. Последняя версия протокола Bitcoin – 70015 – была представлена вместе с основным клиентом Bitcoin 0.13.2. Некоторые узлы предпочитают быть полными узлами с полноценным блокчейном, так как они более безопасны и играют важную роль в распространении, пока другие узлы выполняют только функции сетевой маршрутизации, но не занимаются майнингом или хранением приватных ключей (функционал кошелька). Еще один тип узлов – одиночный майнер, который занимается майнингом, полным хранением блокчейна и выступает узлом маршрутизации.

Bitcoin. Также существует несколько нестандартных, но очень распространенных узлов, которые называются серверы пула протоколов. Эти узлы применяют альтернативные протоколы, такие как stratum. Они могут использоваться в пулах майнинга. Узлы, которые занимаются исключительно вычислением хешей, используют протокол stratum для отправки их решений в пул майнинга. Некоторые узлы выполняют только операции майнинга и называются узлами майнинга. Также возможно запустить SPV-клиент, работающий лишь в режиме кошелька с функцией сетевой маршрутизации без блокчейна. Клиенты SPV могут скачивать только заголовки блоков без синхронизации с сетью, а при необходимости запрашивать транзакции из полных узлов. Верификация транзакций возможна при использовании корня Меркла в заголовке блока с ветвью Меркла для подтверждения присутствия транзакции в блоке блокчейна. Большинство протоколов интернета линейно, что значит, что каждая линия разделена символами возврата каретки (\r) и новой строки (\n). Stratum так же является линейным протоколом, который использует простые TCP-сокеты и читабельный JSON-RPC для взаимодействия и коммуникации между узлами. Stratum обычно применяется для подключения к пулам майнинга. Сеть Bitcoin идентифицируется по различным магическим значениям, как показано на рис. 23.

СЕТЬ	МАГИЧЕСКОЕ ЗНАЧЕНИЕ	HEX
main	0xD9B4BEF9	F9 BE B4 D9
testnet3	0x0709110B	0B 11 09 07

Рис 23 - Магические значения сети Bitcoin

Магические значения используются для индикации исходной сети сообщения. Полный узел выполняет четыре функции: кошелек, майнер, блокчейн и узел сетевой маршрутизации. Прежде чем мы начнем исследовать работу протокола Bitcoin и принцип синхронизации блоков, нам необходимо понять, какие различные типы сообщений использует протокол Bitcoin. Список сообщений предоставлен тут. Суммарно существует 27 типов протоколов, однако вполне вероятен рост их числа с ростом протокола. Далее перечислены наиболее распространенные сообщения протокола их описания:

- Version – это первое сообщение, которое отправляется узлом в сеть, со держит версию и количество блоков. Удаленный узел отвечает аналогичной информацией, и устанавливается соединение;
- Verack – это ответ с версией сообщения для установки соединения; Inv – используется узлами для объявления своего содержимого блоков и транзакций;
- Getdata – это ответ на inv, запрашивающий единичный блок транзакции, идентифицированный его хешем;
- Getblocks – возвращает пакет inv, который содержит список блоков начиная с последнего известного хеша или 500 блоков;
- Getheaders – используется для запроса заголовков блоков в указанном диапазоне;
- T – используется для отправки транзакции в ответ на сообщение протокола getdata; Block – отправляет блок в ответ на сообщение протокола getdata;
- Headers – этот пакет возвращает до 2000 заголовков блоков в ответ на запрос getheaders;
- Getaddr – используется для отправки запроса получения информации об известных пирах;

- Addr – предоставляет информацию об узлах сети. Содержит количество адресов и список адресов в виде IP-адресов и номера порта.

Когда запускается узел Bitcoin, вначале он иницирует исследование всех пиров. Это достигается запросами сидов DNS, которые содержатся в коде клиента Bitcoin и обслуживаются членами сообщества Bitcoin. Этот поиск возвращает количество A-записей DNS. Протокол Bitcoin работает, используя порт TCP 8333 по умолчанию для основной сети и порт TCP 18333 для testnet. Далее вы можете видеть код, который показывает пример сидов DNS в chainparams.cpp:

```
// Pieter Wuille, only supports x1, x5, x9, and xd
vSeeds.emplace_back("seed.bitcoin.sipa.be");
// Matt Corallo, only supports x9
vSeeds.emplace_back("dnsseed.bluematt.me");
// Luke Dashjr vSeeds.emplace_back("dnsseed.bitcoin.dashjr.org");
// Christian Decker, supports x1 - xf vSeeds.emplace_back("seed.bitcoinstats.com");
// Jonas Schnelli, only supports x1, x5, x9, and xd
vSeeds.emplace_back("seed.bitcoin.jonasschnelli.ch");
// Peter Todd, only supports x1, x5, x9, and xd
vSeeds.emplace_back("seed.btc.petertodd.org");
```

Сначала клиент отправляет сообщение протокола version, которое содержит различные поля, такие как версии, службы, временная метка, сетевой адрес, данное время, и некоторые другие поля. Удаленный узел отвечает своим сообщением version, которое сопровождается обменом сообщениями verack между обоими узлами, указывая на успешное соединение.

После этого происходит обмен сообщениями getaddr и addr для поиска пиров, о которых не знает клиент. Тем временем любой из узлов может отправить сообщение ping для проверки активности соединения. Типы сообщений getaddr и addr определены в протоколе Bitcoin. Этот процесс показан на рис. 24

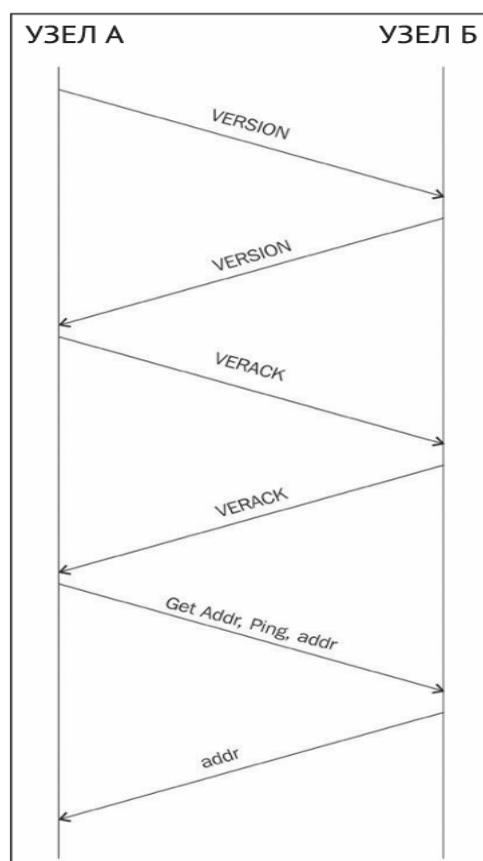


Рис 24 - Визуализация протокола исследования узлов

На рис. 24 показано сообщение между двумя узлами Bitcoin во время изначального соединения. Слева показан УЗЕЛ А, справа – УЗЕЛ Б. Сначала УЗЕЛ А начинает соединение путем отправки сообщения `version` на удаленный пир УЗЕЛ Б, которое содержит номер версии и текущее время. УЗЕЛ Б отвечает своим сообщением `version`, которое также содержит номер версии и текущее время. УЗЕЛ А и УЗЕЛ Б обмениваются сообщениями `verack`, что говорит об успешном соединении. После успешного соединения пиры могут обмениваться сообщениями типа `getaddr` и `addr` для исследования других пиров в сети.

Теперь возможно скачивание блока. Если на узле уже полностью синхронизированы все блоки, тогда начинается прослушивание на наличие новых блоков с применением сообщения протокола `inv`; в ином случае сначала проверяются наличие ответа на сообщения `inv` и наличие инвентаризации. При наличии запрашиваются блоки с применением сообщения протокола `getdata`; при отсутствии запрашивается инвентаризация с помощью сообщения `getblocks`. Этот метод использовался до версии 0.9.3. Это был медленный процесс, известный как подход блоки-сначала, после чего в версии 0.10.0 заменен на подход за головки-сначала.

Процесс скачивания блоков может использовать методы блоки-сначала или заголовки-сначала для синхронизации блоков в зависимости от версии клиента Bitcoin. Первый из методов является медленным, и его поддержка прекратилась с 16 февраля 2015 года с выпуском версии 0.10.0.

С версии 0.10.0 был представлен новый метод скачивания блоков с названием заголовки-сначала. Это многократно улучшило производительность, синхронизация блокчейна стала занимать часы вместо дней, как ранее. Основная идея заключается в том, что новый узел сначала запрашивает заголовки блоков у пиров и подтверждает их. После завершения блоки запрашиваются в параллельной форме со всех доступных пиров, поскольку образ полной цепи уже скачан в виде цепи заголовков блоков.

В этом методе при запуске клиента проверяется, был ли блокчейн целиком синхронизирован, если произведена синхронизация цепи заголовков; если нет, что является случаем первого запуска клиента, запрашиваются заголовки от других пиров с применением сообщения `getheaders`. Если блокчейн целиком синхронизирован, он прослушивает новые блоки через сообщения `inv`. Если цепь заголовков уже целиком синхронизирована, будут запрошены блоки с применением сообщения протокола `getdata`. Также узел проверяет, имеет ли цепь заголовков больше заголовков, чем блоков, после чего запрашивает блоки с применением сообщения протокола `getdata`.

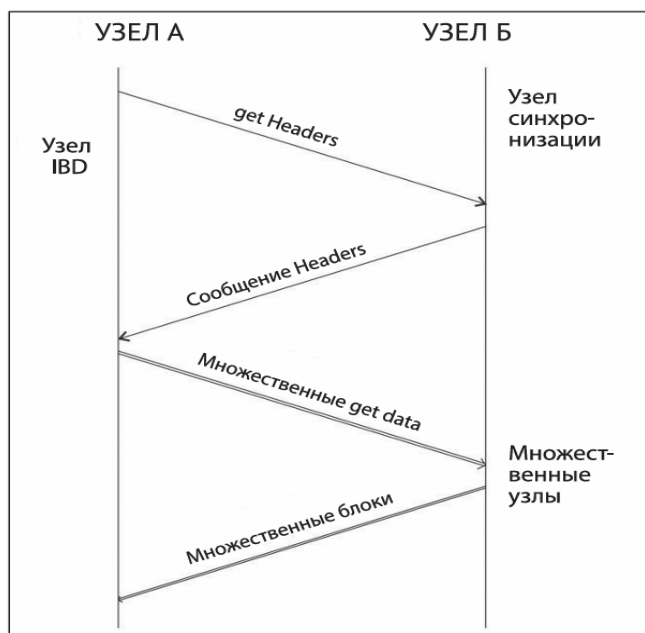


Рис 25 - Синхронизация заголовков и блоков в основном клиенте Bitcoin версии $\geq 0.10.0$

На рис. 25 показан процесс синхронизации блоков Bitcoin между двумя узлами сети Bitcoin. УЗЕЛ А в левой части схемы называется Узлом IBD (Initial Block Download Node – исходный узел загрузки блоков), а УЗЕЛ Б в правой части называется Узлом синхронизации. Узел IBD является тем узлом, который запрашивает блоки, а Узел синхронизации – узлом, из которого они запрашиваются. Процесс начинается с отправки УЗЛОМ А сообщения `getheaders`, которое получает ответ в виде сообщения `headers` от Узла синхронизации. Рабочие данные сообщения `getheaders` – это один или более хешей заголовков. Если это новый узел, то в нем присутствует только хеш заголовка первого блока генезиса. Узел синхронизации отвечает отправкой до 2000 заголовков блоков Узлу IBD. После этого Узел IBD начинает скачивать больше заголовков из Узла синхронизации, параллельно скачивая блоки из различных узлов. Иными словами, Узел IBD отправляет запросы раз личным узлам и в результате получает различные блоки от Узла синхронизации и других узлов. Если на Узле синхронизации нет больше 2000 заголовков во время запроса `getheaders` от Узла IBD, тогда последний отправляет запросы `getheaders` на другие узлы. Этот процесс продолжается в параллели, пока не будет выполнена синхронизация блокчейна.

Вызовы `getblockchaininfo` и `getpeerinfo` получили обновленный функционал для учета этих изменений. Удаленный вызов процедур (Remote Procedure Call – RPC) `getchaintips` используется для перечня всех известных ветвей блок чейна, включая блоки, содержащие только заголовки. Вызов `getblockchaininfo` используется для предоставления информации о текущем состоянии блокчейна. Вызов `getpeerinfo` используется для перечня количества блоков и заголовков, являющихся общими между пирами. Wireshark может использоваться для визуализации обмена сообщениями между пирами, а также служить бесценным инструментом для изучения протокола Bitcoin. На рис. 26 вы можете наблюдать образец.

Это базовый пример, демонстрирующий сообщения version, verack, getaddr, ping, addr и inv. В деталях вы можете видеть важную информацию, такую как тип пакетов, название команды и результаты сообщений протокола.

Filter: ip.dst == 52.1.165.219 and bitcoin Expression... Clear Apply Save						
No.	Time	Source	Destination	Protocol	Length	Info
131	98.598526000	192.168.0.13	52.1.165.219	Bitcoin	192	version
150	99.180294000	192.168.0.13	52.1.165.219	Bitcoin	90	verack
151	99.180421000	192.168.0.13	52.1.165.219	Bitcoin	122	getaddr, ping
152	99.180715000	192.168.0.13	52.1.165.219	Bitcoin	1288	addr, getheaders[Malformed Packet]
486	112.053746000	192.168.0.13	52.1.165.219	Bitcoin	127	inv
818	143.630367000	192.168.0.13	52.1.165.219	Bitcoin	127	inv
1004	178.729768000	192.168.0.13	52.1.165.219	Bitcoin	127	inv

Transmission Control Protocol, Src Port: 52864 (52864), Dst Port: 18333 (18333), Seq: 207, Ack: 1291, Len: 1222						
Bitcoin protocol						
Packet magic: 0x0b110907						
Command name: addr						
Payload Length: 31						
Payload checksum: 0xa03fc07d						
Address message						
Count: 1						
Address: afbd025800ffff...						
Node services: 0x0000000000000000						
...0 = Network node: Not set						
Node address: ::ffff:86.15.44.209 (::ffff:86.15.44.209)						
Node port: 18333						
Address timestamp: Oct 16, 2016 00:37:19.000000000 BST						
Bitcoin protocol						
Packet magic: 0x0b110907						
Command name: getheaders						
Payload Length: 1029						
Payload checksum: 0x4e54961d						
Getheaders message						
Count: 126						
Starting hash: 1101001f152142abccc039503abc56b149bd56c2b3925b65...						
Starting hash: 000000001980703bd53b0c7bf0ac995bccfeffd5cddc780...						
Starting hash: 000000007ad1fed813d20301b1762895a2e5b08c8a58b3ea...						
Starting hash: 000000003624c451f726a3e983d02279d9c7cf672d36f1d5...						

Рис 26 - Образец сообщения блока в Wireshark

Рисунок 26 демонстрирует поток данных между двумя пирами. Благодаря этому вы можете понять, когда запускается узел и какие типы сообщений используются. В следующем примере диссектор Bitcoin применяется для анализа трафика и идентификации команд протокола Bitcoin. Обмен такими сообщениями, как version, getaddr и getdata, может быть виден на следующем примере вместе с соответствующим комментарием с описанием названия сообщения. Это упражнение может быть полезным для изучения протокола Bitcoin, проведение экспериментов рекомендуется в Bitcoin testnet (более деталь но по ссылке: en.bitcoin.it/wiki/Testnet), в котором различные сообщения и транзакции могут быть отправлены по сети, после чего проанализированы в Wireshark.

На рис. 27 вы можете наблюдать анализ, выполненный в Wireshark, который демонстрирует обмен сообщениями между двумя узлами. Если вы при смотрите внимательно, то увидите протокол исследования узлов, о котором говорилось ранее.

Time	192.168.0.13	136.243.139.96	Comment
97.734135000	(5768) → (18333)	version	Bitcoin: version
98.025045000	(5768) → (18333)	verack	Bitcoin: verack
98.025177000	(5768) → (18333)	getaddr, ping, addr	Bitcoin: getaddr, ping, addr
98.025468000	(5768) → (18333)	getheaders	Bitcoin: getheaders, [unknown command], [unknown command], [unknown command], headers
98.160419000	(5768) → (18333)	TCP Retran	Bitcoin: [TCP Retransmission], getheaders, [unknown command], [unknown command], [unknown command]
98.598399000	(5768) → (18333)	getdata	Bitcoin: getdata
144.343544000	(5768) → (18333)	inv	Bitcoin: inv
176.152240000	(5768) → (18333)	getdata	Bitcoin: getdata
179.493755000	(5768) → (18333)	getdata	Bitcoin: getdata
218.101646000	(5768) → (18333)	ping	Bitcoin: ping
218.192004000	(5768) → (18333)	[unknown.co]	Bitcoin: [unknown command]
218.444431000	(5768) → (18333)	TCP Retran	Bitcoin: [TCP Retransmission], [unknown command]
336.234936000	(5768) → (18333)	getdata	Bitcoin: getdata
337.843423000	(5768) → (18333)	[unknown.co]	Bitcoin: [unknown command]
338.143885000	(5768) → (18333)	ping	Bitcoin: ping
448.764093000	(5768) → (18333)	getdata	Bitcoin: getdata
457.894823000	(5768) → (18333)	[unknown.co]	Bitcoin: [unknown command]
458.195265000	(5768) → (18333)	ping	Bitcoin: ping
578.011774000	(5768) → (18333)	[unknown.co]	Bitcoin: [unknown command]
578.212044000	(5768) → (18333)	ping	Bitcoin: ping
585.587671000	(5768) → (18333)	inv	Bitcoin: inv
647.169633000	(5768) → (18333)	inv	Bitcoin: inv
671.962545000	(5768) → (18333)	getdata	Bitcoin: getdata
698.037067000	(5768) → (18333)	[unknown.co]	Bitcoin: [unknown command]
698.237350000	(5768) → (18333)	ping	Bitcoin: ping
701.563581000	(5768) → (18333)	inv	Bitcoin: inv
701.986269000	(5768) → (18333)	inv	Bitcoin: inv
705.022173000	(5768) → (18333)	inv	Bitcoin: inv
812.115878000	(5768) → (18333)	inv	Bitcoin: inv
818.198570000	(5768) → (18333)	[unknown.co]	Bitcoin: [unknown command]
818.298733000	(5768) → (18333)	ping	Bitcoin: ping

Рис 27 - Протокол исследования узлов в Wireshark

Полные клиенты, или полные узлы – это узлы, которые скачали весь блокчейн; это самый безопасный способ подтверждения блокчейна на уровне клиента. Узлы сети Bitcoin могут работать в двух основных режимах: полный клиент или легкий SPV-клиент. SPV-клиенты используются для подтверждения платежей без необходимости скачивания всего блокчейна. Они только хранят копию заголовков блоков текущего подтвержденного и самого длинного блокчейна. Верификация производится путем поиска ветви Меркла, которая связывает транзакции с исходным блоком, в котором была принята транзакция. Это не очень практично и требует более практичного подхода, который был внедрен с BIP 37 (более детально – по ссылке github.com/bitcoin/bips/blob/master/bip-0037.mediawiki), где для фильтрации релевантных транзакций использовались фильтры Блума.

Фильтр Блума является структурой данных (битовый вектор с индексами), который используется для тестирования членства элемента вероятностным методом. На базовом уровне он предоставляет вероятностный поиск с ошибками включения, но без ошибок исключения. Это значит, что данный фильтр может производить вывод, в котором элемент, не являющийся членом тестируемого набора, ошибочно считается находящимся в наборе, но никогда не произведет вывод, в котором элемент существует в наборе, но утверждается обратное.

Элементы добавляются в фильтр Блума после многократного хеширования, после чего соответствующие биты в битовом векторе устанавливаются в значение 1 через соответствующий индекс. Для проверки присутствия элемента в фильтре Блума применяются те же функции хеширования и сравниваются с битами в битовом векторе для сверки, установлены ли те же биты в значение 1. Для фильтра Блума подходит не любая функция хеширования (такая как SHA-1), поскольку они должны быть быстрыми, независимыми и равномерно распределены. Наиболее распространенными функциями хеширования для фильтров Блума являются FNV, Murmur и Jenkins. Эти фильтры обычно используются клиентами SPV для запроса транзакций и блоков Меркла, в которых они заинтересованы. Блок Меркла – это легковесная версия блока, которая включает в себя заголовок блока, некоторые хеши, список 1-битовых флагов и счетчик транзакций. Эта информация может далее применяться для построения дерева Меркла, что достигается путем создания фильтра, который соответствует только тем транзакциям и блокам, которые были запрошены SPV-клиентом. Как только произошел обмен сообщениями version и установлена связь между пирами, узлы могут настраивать фильтры согласно своим требованиям. Эти вероятностные фильтры предоставляют широкий градус приватности или

точности в зависимости от того, насколько свободно они были настроены. Строгий фильтр Блума будет фильтровать только те транзакции, которые были запрошены узлом, жертвуя возможностью раскрытия обращений пользователей к оппонентам, которые могут коррелировать транзакции с их IP-адресами, тем самым компрометируя приватность. С другой стороны, свободно настроенный фильтр может привести к получению большего числа нерелевантных транзакций в обмен на больший уровень приватности. Так же для SPV-клиентов фильтры Блума разрешают работу с низким расходом пропускной способности сети, по сравнению со скачиванием всех транзакций для верификации. BIP 37 предложил внедрение фильтров Блума в Bitcoin и представил три новых сообщения для протокола Bitcoin: Filterload. Используется для установки фильтра Блума при подключении; Filteradd. Добавляет новый элемент данных к текущему фильтру; Filterclear. Удаляет текущий фильтр.

Среда разработки Ethereum

Первым делом нужно сконфигурировать среду разработки. В этом разделе представлена конфигурация для тестовых и частных сетей Ethereum. Тестовая сеть называется Ropsten и применяется разработчиками и пользователями в качестве экспериментальной площадки для проверки смарт-контрактов и других технологий, которые со временем могут стать частью блокчейна. Частная сеть позволяет создать общий распределенный журнал, с помощью которого разработчики могут тестировать свои смарт-контракты. В предыдущей главе мы рассматривали разные клиенты для Ethereum, такие как Parity, но самым распространенным из них является Geth, поэтому в наших примерах мы будем использовать именно его.

Разработка для блокчейна Ethereum может осуществляться разными способами. В этой главе мы обсудим все общепринятые варианты. Мы покроем следующие темы:

- тестовые сети;
- конфигурация частной сети Ethereum.

Разрабатывать и тестировать смарт-контракты в Ethereum можно по-разному. Обычно для этого используют локальную частную сеть или виртуальную среду. На следующем этапе происходит развертывание в публичной тестовой сети. После успешного проведения всех тестов контракты можно развернуть в основной или частной промышленной сети. У некоторых разработчиков этот процесс может немного отличаться. Например, окончательное развертывание может происходить сразу после тестирования в локальной виртуальной среде. Это ускоряет разработку, так как подготовка локального симулятора блокчейна обычно занимает меньше времени, чем конфигурация частной сети. Все эти нюансы будут рассмотрены в главах 13 и 14.

Чтобы упростить разработку и тестирование в Ethereum, можно воспользоваться такими новыми инструментами, как Truffle, Ganache и MetaMask. Мы подробно обсудим их в следующей главе, а пока что разработка смарт-контрактов и развертывание их в локальной сети будут осуществляться вручную, с помощью командной строки. Это позволит вам понять, что на самом деле происходит «под капотом».

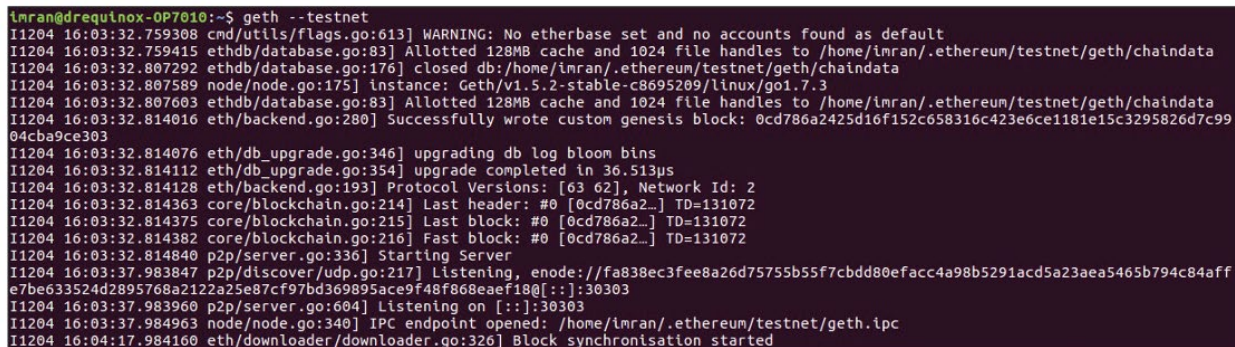
Фреймворки и инструменты облегчают разработку, но скрывают от вас большинство деталей, без понимания которых новичкам будет сложно усвоить базовые принципы. Внутренние подробности – это тоже важно. Поэтому вначале мы научимся разрабатывать с помощью стандартных средств, доступных в Ethereum, и только потом, овладев фундаментальными знаниями, перейдем к использованию таких фреймворков, как Truffle, которые делают разработку и тестирование крайне простыми.

Тестовые Сети

Клиент Geth, написанный на языке Go (geth.ethereum.org), позволяет подключиться к тестовой сети Ethereum с помощью следующей команды:

```
$ geth --testnet
```

На рис. 28 показан примерный вывод, который вы получите. Здесь можно видеть тип выбранной сети и различные сведения о загрузке блокчейна.

The image shows a terminal window with a dark background and light-colored text. It displays the output of the command 'geth --testnet'. The output consists of several lines of log messages, including warnings about missing etherbase and accounts, database initialization, cache allocation, genesis block writing, database upgrade, protocol version reporting, and server listening. The messages are timestamped and include file paths and technical details related to the Geth client's startup process.

```
imran@drequinox-0P7010:~$ geth --testnet
I1204 16:03:32.759308 cmd/utlils/flags.go:613] WARNING: No etherbase set and no accounts found as default
I1204 16:03:32.759415 ethdb/database.go:83] Allotted 128MB cache and 1024 file handles to /home/imran/.ethereum/testnet/geth/chaindata
I1204 16:03:32.807292 ethdb/database.go:176] closed db:/home/imran/.ethereum/testnet/geth/chaindata
I1204 16:03:32.807589 node/node.go:175] instance: Geth/v1.5.2-stable-c8695209/linux/go1.7.3
I1204 16:03:32.807603 ethdb/database.go:83] Allotted 128MB cache and 1024 file handles to /home/imran/.ethereum/testnet/geth/chaindata
I1204 16:03:32.814016 eth/backend.go:280] Successfully wrote custom genesis block: 0cd786a2425d16f152c658316c423e6ce1181e15c3295826d7c99
04cba9ce303
I1204 16:03:32.814076 eth/db_upgrade.go:346] upgrading db log bloom bins
I1204 16:03:32.814112 eth/db_upgrade.go:354] upgrade completed in 36.513µs
I1204 16:03:32.814128 eth/backend.go:193] Protocol Versions: [63 62], Network Id: 2
I1204 16:03:32.814363 core/blockchain.go:214] Last header: #0 [0cd786a2...] TD=131072
I1204 16:03:32.814375 core/blockchain.go:215] Last block: #0 [0cd786a2...] TD=131072
I1204 16:03:32.814382 core/blockchain.go:216] Fast block: #0 [0cd786a2...] TD=131072
I1204 16:03:32.814840 p2p/server.go:336] Starting Server
I1204 16:03:37.983847 p2p/discover/udp.go:217] Listening, enode://fa838ec3fee8a26d75755b55f7cbdd80efacc4a98b5291acd5a23aea5465b794c84aff
e7be633524d2895768a2122a25e87cf97bd369895ace9f48f868eaeaf180[:]:30303
I1204 16:03:37.983960 p2p/server.go:604] Listening on [::]:30303
I1204 16:03:37.984963 node/node.go:340] IPC endpoint opened: /home/imran/.ethereum/testnet/geth.ipc
I1204 16:04:17.984160 eth/downloader/downloader.go:326] Block synchronisation started
```

Рис 28 - Вывод команды geth при подключении к тестовой сети Ethereum

На сайте ropsten.etherscan.io можно отслеживать транзакции и блоки в тестовой сети. Помимо Ropsten, существуют и другие тестовые сети, такие как Frontier, Morden и Rinkeby. Для подключения к ним можно использовать флаги командной строки:

testnet: Ropsten network: pre-configured proof-of-work test network

--rinkeby: Rinkeby network: pre-configured proof-of-authority test network

--networkid value: Network identifier (integer, 1=Frontier, 2=Morden (disused), 3=Ropsten, 4=Rinkeby) (default: 1)

Подготовка Частной Сети

В частной сети можно создать совершенно новый блокчейн, который, в отличие от основной и тестовой сетей, использует собственный начальный блок и сетевой идентификатор. Для этого нам понадобится три компонента:

- идентификатор сети;
- файл с начальным блоком;
- директория для хранения данных блокчейна. Этот параметр не является обязательным. Его следует указывать только в случае, если в системе уже есть другие активные блокчейны и вы хотите хранить новую цепочку в отдельном месте.

В основной сети Ethereum клиент Geth способен обнаруживать загрузочные узлы, так как они прописаны в его коде. Это позволяет ему подключаться автоматически. Но в случае с частной сетью вам необходимо указать подходящие флаги и конфигурацию, чтобы ваш клиент мог обнаруживать другие узлы и быть доступным для обнаружения. Вскоре мы увидим, как это делается. Помимо настройки трех вышеупомянутых компонентов, желательно отключить обнаружение узлов. Это сделает вашу частную сеть более безопасной и не даст внешним интернет-узлам случайно к ней подключиться. Случайное подключение может произойти, если другие сети используют такой же идентификатор и файл с начальным блоком; и хотя вероятность этого очень низкая, отключение обнаружения узлов считается хорошей идеей.

Идентификатор сети

Сетевым идентификатором может быть любое положительное число; исключения составляют числа 1 и 3, которые используются в основной и, соответственно, тестовой (Ropsten) сетях Ethereum. Для частной сети, которая будет представлена позже в этом разделе, был выбран идентификатор 786.

Файл с начальным блоком

Этот файл содержит поля, необходимые для описания произвольного начального блока. Это первый блок в сети, и у него нет родителя. Протокол Ethereum следит за тем, чтобы в процессе консенсуса могли принимать участие только те узлы, которые имеют тот же начальный блок. Для идентификации сети обычно указывают идентификатор цепочки. Ниже показан файл с начальным блоком, который будет использоваться в нашем примере:

```
{
  "nonce":      "0x00000000000000042",      "timestamp":      "0x00",      "parentHash":
  "0x0000000000000000000000000000000000000000000000000000000000000000",
  "extraData":  "0x00",  "gasLimit":  "0x8000000",  "difficulty":  "0x0400",  "mixhash":
  "0x0000000000000000000000000000000000000000000000000000000000000000",  "coinbase":
  "0x3333333333333333333333333333333333333333333333333333333",  "alloc": { },  "config": { "chainId": 786,
  "homesteadBlock": 0, "eip155Block": 0, "eip158Block": 0 } }
```

Этот код следует сохранить в текстовый файл с расширением JSON; напри мер, privategenesis.json. Вы также можете заранее выделить Ether, указав адрес бенефициара и количество Wei, но делать это необязательно, так как майнинг в частной сети происходит очень быстро.

Если вы все же хотите выделить определенные средства, добавьте в свой файл следующий код:

```
"alloc": { "0xcf61d213faa9acadbf0d110e1397caf20445c58f": { "balance": "100000" }, }
```

Теперь давайте пройдемся по каждому из этих параметров.

nonce. Это 64-битный хеш, который используется в качестве доказательства выполнения достаточного объема работы (PoW). Он указывается в сочетании с параметром mixhash. timestamp. Это временная отметка блока в формате Unix. Используется при проверке порядка следования блоков и для регулирования сложности. На пример, если блок генерируется слишком быстро, сложность возрастает.

parentHash. У начального блока нет родителей, поэтому данный параметр всегда равен нулю.

extraData. Этот параметр позволяет сохранить вместе с блоком произвольное 32-битное значение.

gasLimit. Это лимит на расход газа для каждого отдельного блока.

difficulty. Определяет цель майнинга и представляет собой уровень сложности хеша, который требуется для доказательства PoW.

mixhash. Это 256-битный хеш, который в сочетании с nonce является доказательством расходования достаточного количества вычислительных ресурсов (для удовлетворения PoW).

coinbase. Это 160-битный адрес, на который будет отправлена награда за успешный майнинг.

alloc. Список предварительно выделенных кошельков. Длинное шестнадцатеричное число представляет учетную запись, которой назначается баланс.

config. Данный раздел содержит различную конфигурационную информацию, которая определяет идентификатор цепочки и номера блоков для жесткого ответвления блокчейна. В частных сетях этот параметр можно опустить.

Директория с данными

Это директория, в которой будут храниться данные блокчейна для частной сети Ethereum. В нашем примере это ~/etherprivate/. Для запуска и тонкой настройки клиента Geth, а также для поднятия частной сети необходимо указать ряд параметров.

Флаги и их назначение

Ниже перечислены флаги, которые поддерживаются клиентом Geth:

--nodiscover. Исключает автоматическое обнаружение узла в случае использования идентичного начального блока и идентификатора сети;

--maxpeers. Определяет максимальное количество узлов, которые могут подключиться к частной сети. Если указать 0, подключение станет невозможным; это может пригодиться в некоторых случаях – например, при закрытом тестировании;

--rpc. Позволяет включить в Geth интерфейс RPC;

--rpcapi. Список разрешенных API-интерфейсов. Например, если указать eth, web3, в RPC станут доступными интерфейсы Eth и Web3;

--rpcport. TCP-порт для RPC; например, 9999;

--rpccorsdomain. URL-адрес, с которого позволено подключаться к частному узлу Geth и выполнять операции по RPC. cors в --rpccorsdomain расшифровывается как cross-origin resource sharing (совместное использование ресурсов между разными источниками);

--port. TCP-порт, на котором будут отслеживаться входящие подключения с других узлов;

--identity. Строка, определяющая название частной сети.

Статические узлы

При необходимости вы можете подключиться к определенным узлам. Для этого их адреса нужно сохранить в файл с названием static-nodes.json, который находится в одной директории с файлами chaindata и keystore (в нашем примере это ~/etherprivate/). Это позволяет обнаруживать узлы в частных сетях. Пример JSON-файла показан ниже:

```
"enode://  
44352ede5b9e792e437c1c0431c1578ce3676a87e1f588434aff1299d30325c233c8d426fc5  
7a25380481c8a36fb3be2787375e932fb4885885f6452f6efa77f@xxx.xxx.xxx.xxx:TCP_PORT" ]
```

xxx – это публичный IP-адрес, а в качестве TCP_PORT можно указать корректный TCP-порт, доступный в системе. Длинная шестнадцатеричная строка – это идентификатор узла.

Запуск частной сети

Ниже показана начальная команда для запуска частной сети:

```
$ ./geth init ~/ethpriv/privategenesis.json --datadir ~/ethpriv/
```

Предполагается, что в домашней директории находится каталог etherprivate с файлом privategenesis.json внутри.

Примерный вывод этой команды представлен на рис. 29

```
WARN [12-13|19:19:11] No etherbase set and no accounts found as default
INFO [12-13|19:19:11] Allocated cache and file handles      database=/Users/drequinox/etherprivate/gets/chaindata cache=16 handles=16
INFO [12-13|19:19:11] Writing custom genesis block
INFO [12-13|19:19:11] Successfully wrote genesis state        database=chaindata hash=6650a0..b5c158
INFO [12-13|19:19:11] Allocated cache and file handles      database=/Users/drequinox/etherprivate/gets/lightchaindata cache=16 handles=16
INFO [12-13|19:19:11] Writing custom genesis block
INFO [12-13|19:19:11] Successfully wrote genesis state        database=lightchaindata hash=6650a0..b5c158
```

Рис 29 - Инициализация частной сети

Судя по всему, начальный блок успешно создан. Запустить клиента geth можно с помощью такой команды:

```
$ ./geth --datadir ~/etherprivate/ --networkid 786 --rpc --rpcapi 'web3,eth,net,debug,personal' --rpccorsdomain '*'
```

В ответ мы получим следующий вывод (рис. 30).

```
WARN [12-13|19:20:11] No etherbase set and no accounts found as default
INFO [12-13|19:20:11] Starting peer-to-peer node             instance=Geth/v1.7.3-stable-4bb3c89d/darwin-amd64/go1.9.2
INFO [12-13|19:20:11] Allocated cache and file handles      database=/Users/drequinox/etherprivate/gets/chaindata cache=16 handles=16
WARN [12-13|19:20:11] Upgrading database to use lookup entries
INFO [12-13|19:20:11] Initialised chain configuration        config="{ChainID: 786 Homestead: 0 DAO: <nil> DAOsupport:
INFO [12-13|19:20:11] Disk storage enabled for ethash caches dir=/Users/drequinox/etherprivate/gets/ethash count=3
INFO [12-13|19:20:11] Disk storage enabled for ethash DAGs  dir=/Users/drequinox/.ethash count=2
INFO [12-13|19:20:11] Initialising Ethereum protocol        versions="[63 62]" network=786
INFO [12-13|19:20:11] Database deduplication successful      deduped=0
INFO [12-13|19:20:11] Loaded most recent local header        number=0 hash=6650a0..b5c158 td=1024
INFO [12-13|19:20:11] Loaded most recent local full block    number=0 hash=6650a0..b5c158 td=1024
INFO [12-13|19:20:11] Loaded most recent local fast block    number=0 hash=6650a0..b5c158 td=1024
INFO [12-13|19:20:11] Regenerated local transaction journal  transactions=0 accounts=0
INFO [12-13|19:20:11] Starting P2P networking
INFO [12-13|19:20:13] UDP listener up                       self=enode://5c53ec0755806bc92432728f22a55f169a8c63df307fc
7a55d635@86.15.44.209:30303
INFO [12-13|19:20:13] RLPx listener up                     self=enode://5c53ec0755806bc92432728f22a55f169a8c63df307fc
7a55d635@86.15.44.209:30303
INFO [12-13|19:20:13] IPC endpoint opened: /Users/drequinox/etherprivate/gets.ipc
INFO [12-13|19:20:14] Mapped network port                  proto=udp extport=30303 intport=30303 interface="UPNP IGDv
INFO [12-13|19:20:14] Mapped network port                  proto=tcp extport=30303 intport=30303 interface="UPNP IGDv
```

Рис 30 - Запуск geth для подключения к частной сети

Теперь мы можем подключить наш клиент к экземпляру geth, запущенному в частной сети. Для этого используется механизм межпроцессного взаимодействия (англ. Inter-Process Communications, или IPC), который позволяет общаться двум процессам, размещенным в одной локальной системе. Выполним следующую команду:

```
$ geth attach ipc:.ethereum/privatenet/geth.ipc
```

Как видно по рис. 31, эта команда откроет интерактивную консоль Java Script для текущего сеанса частной сети.

```
Welcome to the Geth JavaScript console!

instance: Geth/v1.7.3-stable-4bb3c89d/darwin-amd64/go1.9.2
modules: admin:1.0 debug:1.0 eth:1.0 miner:1.0 net:1.0 personal:1.0 rpc:1.0 txpool:1.0 web3:1.0
>
```

Рис 31 - Интерактивная консоль Java Script

Подключение geth к частной сети 786

В верхней строчке на рис. 12.4 можно заметить предупреждение, которое выводится при запуске Geth. Оно переводится следующим образом:

В не установлена учетная запись etherbase и не выбрана учетная запись по умолчанию.

Мы видим это сообщение, поскольку в нашей новой тестовой сети не назначена главная учетная запись (etherbase), которая будет получать вознаграждения за майнинг. Чтобы решить эту проблему, достаточно создать новую учетную запись и пометить ее как etherbase. Это необходимо сделать перед тем, как начинать майнинг в тестовой сети. Команды для этого перечислены ниже. Заметьте, что вводить их нужно в консоли JavaScript клиента Geth, как это показано на рис. 12.4. Следующая команда создает новую учетную запись в частной сети с идентификатором 786:

```
personal.newAccount("Password123")
"0xc61d213faa9acadbf0d110e1397caf20445c58f"
```

После создания учетной записи ее следует пометить как `etherbase/coinbase`, чтобы ей начислялась награда за майнинг. Для этого предусмотрена следующая команда:

```
> miner.setEtherbase(personal.listAccounts[0])
true
```

Пока что на балансе учетной записи etherbase нет средств. Вы сами можете в этом убедиться:

```
> eth.getBalance(eth.coinbase).toNumber();
0
```

Наконец, мы можем начать майнинг. Команда, представленная ниже, при нимает один параметр – количество потоков. В следующем примере мы выде ляем для процесса майнинга два потока, передавая функции `start` аргумент 2:

```
> miner.start(2)
true
```

После запуска майнинга начинается генерация DAG, в результате чего получается примерно такой вывод (рис. 32):

[illegible]

Рис 32 - Генерация DAG

После окончания генерации DAG и начала майнинга клиент Geth выводит информацию, показанную на рис. 33. Сообщение `Mined 5 blocks back: . . .` однозначно говорит о том, что майнинг блоков происходит успешно.

```
I1204 22:38:02.373804 miner/worker.go:438] ⚡ Mined 5 blocks back: block #487
I1204 22:38:02.373908 miner/worker.go:542] commit new work on block 493 with 0 txs & 0 uncles. Took 86.005µs
I1204 22:38:02.637297 miner/worker.go:344] ⚡ Mined block (#493 / 9a95245e). Wait 5 blocks for confirmation
I1204 22:38:02.637415 miner/worker.go:542] commit new work on block 494 with 0 txs & 0 uncles. Took 91.009µs
I1204 22:38:02.637436 miner/worker.go:438] ⚡ Mined 5 blocks back: block #488
I1204 22:38:02.639064 miner/worker.go:542] commit new work on block 494 with 0 txs & 0 uncles. Took 1.609044ms
I1204 22:38:03.538525 miner/worker.go:344] ⚡ Mined block (#494 / cb89ccdd). Wait 5 blocks for confirmation
I1204 22:38:03.538719 miner/worker.go:542] commit new work on block 495 with 0 txs & 0 uncles. Took 158.751µs
I1204 22:38:03.538745 miner/worker.go:438] ⚡ Mined 5 blocks back: block #489
I1204 22:38:03.538860 miner/worker.go:542] commit new work on block 495 with 0 txs & 0 uncles. Took 95.822µs
I1204 22:38:03.548923 miner/worker.go:344] ⚡ Mined block (#495 / 539d8079). Wait 5 blocks for confirmation
I1204 22:38:03.549064 miner/worker.go:542] commit new work on block 496 with 0 txs & 0 uncles. Took 120.447µs
I1204 22:38:03.549082 miner/worker.go:438] ⚡ Mined 5 blocks back: block #490
I1204 22:38:03.549159 miner/worker.go:542] commit new work on block 496 with 0 txs & 0 uncles. Took 64.047µs
```

Рис 33 - Вывод майнинга

Для остановки майнинга предусмотрена следующая команда:

```
> miner.stop
```

True

В консоли JavaScript можно посмотреть общий текущий баланс Ether (см. следующий листинг). Как видите, мы сгенерировали существенную сумму. В частных сетях, в которых отсутствует конкуренция при решении PoW, скорость майнинга очень высокая. К тому же мы выбрали довольно низкую сложность при описании начального блока.

```
> eth.getBalance(eth.coinbase).toNumber();
```

```
8500000000000000000
```

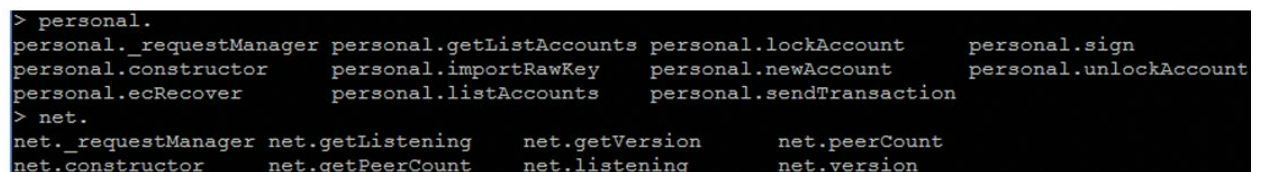
Чтобы вывести список доступных объектов, нужно последовательно по два раза нажать клавиши Пробел и Tab. Результат можно видеть на рис. 34.



```
>
Array      Math      TypeError  constructor  hasOwnProperty  parseFloat  toString
BigNumber  NaN      URIError   debug        inspect         parseInt   txpool
Boolean    Number   Web3       decodeURI    isFinite        personal   undefined
Date       Object   _setInterval  decodeURIComponent  isNaN          propertyIsEnumerable  unescape
Error      RangeError  _setTimeout  encodeURI     isPrototypeOf   require    valueOf
EvalError  ReferenceError  admin        encodeURIComponent  jeth           rpc        web3
Function   RegExp    clearInterval  escape        loadScript      setInterval
Infinity   String    clearTimeout   eth           miner           setTimeout
JSON       SyntaxError  console      eval          net             toLocaleString
```

Рис 34 - Доступные объекты

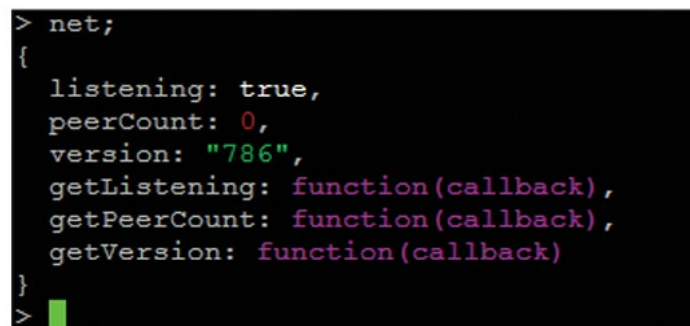
Кроме того, во время ввода команды можно дважды начать Tab, чтобы получить автодополнение. В итоге появится список доступных методов, как по казано на рис. 35.



```
> personal.
personal._requestManager  personal.getListAccounts  personal.lockAccount  personal.sign
personal.constructor      personal.importRawKey     personal.newAccount   personal.unlockAccount
personal.ecRecover        personal.listAccounts     personal.sendTransaction
> net.
net._requestManager  net.getListening  net.getVersion  net.peerCount
net.constructor      net.getPeerCount  net.listening   net.version
```

Рис 35 - Доступные методы

Список методов, доступных в объекте, можно также получить, набрав после команды точку с запятой (;). В примере на рис. 36 показан список всех методов объекта net.



```
> net;
{
  listening: true,
  peerCount: 0,
  version: "786",
  getListening: function(callback),
  getPeerCount: function(callback),
  getVersion: function(callback)
}
>
```

Рис 36 - Список методов

Есть еще несколько команд, которые можно применить по отношению к частной сети. Например:

получение текущей цены газа:

```
> eth.gasPrice
```

Under Mac OS this is usually available under /Applications/Ethereum Wallet.app/Contents/MacOS

Подключиться к geth из Mist можно двумя способами: через IPC или RPC/ HTTP. Соответствующие команды показаны ниже:

\$./Ethereum Wallet --rpc ~/.ethereum/privatenet/geth.ipc

\$./Ethereum Wallet --rpc <http://127.0.0.1:8545>

Если запустить Mist с флагом -rpc и HTTP-адресом в качестве значения, на экране появится предупреждение о том, что это менее безопасный метод соединения. Просто нажмите кнопку ОК. Это частная тестовая сеть, которая работает локально, не подключена к интернету и не будет использоваться извне – следовательно, никаких проблем быть не должно.

Сообщение будет выглядеть следующим образом (рис. 37).

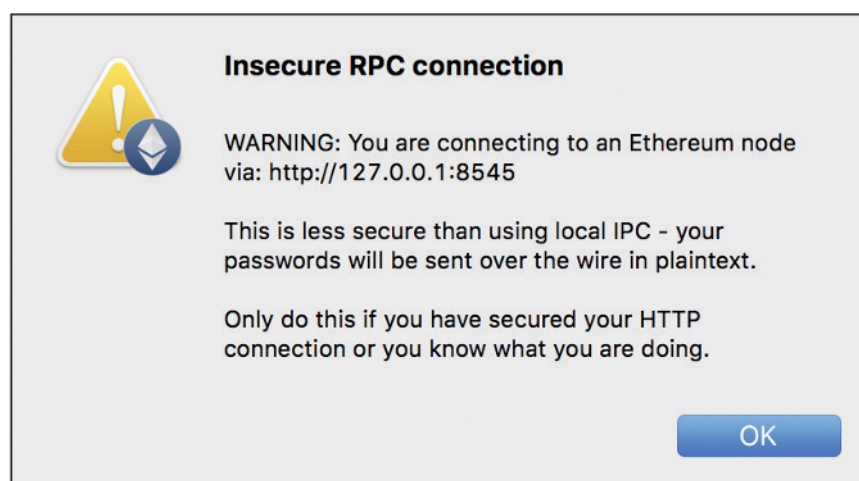


Рис 37 - Небезопасное RPC-соединение

Это позволит подключиться к сеансу Geth в частной сети и получить все необходимые возможности Mist: кошелек, управление учетными записями и развертывание контрактов (рис. 38).

```
imran@drequinox-OP7010: /opt/Ethereum Wallet
imran@drequinox-OP7010: /opt/Ethereum Wallet$ ./Ethereum\ Wallet --rpc /home/imran/.ethereum/privatenet/geth.ipc
[2016-12-06 07:58:08.706] [INFO] main - Running in production mode: true
Secp256k1 bindings are not compiled. Pure JS implementation will be used.
[2016-12-06 07:58:08.860] [INFO] main - Starting in Wallet mode
[2016-12-06 07:58:08.932] [INFO] Db - Loading db: /home/imran/.config/Ethereum Wallet/mist.lokidb
[2016-12-06 07:58:08.947] [INFO] Windows - Creating commonly-used windows
[2016-12-06 07:58:08.948] [INFO] Windows - Create secondary window: loading, owner: notset
[2016-12-06 07:58:09.612] [INFO] updateChecker - Check for update...
[2016-12-06 07:58:11.373] [INFO] Windows - Create primary window: main, owner: notset
[2016-12-06 07:58:11.385] [INFO] Windows - Create primary window: splash, owner: notset
[2016-12-06 07:58:11.989] [INFO] ipcCommunicator - Backend language set to: en-GB
[2016-12-06 07:58:13.199] [INFO] (ui: splash) - Web3 already initialized, re-using provider.
[2016-12-06 07:58:13.362] [INFO] ClientBinaryManager - Initializing...
[2016-12-06 07:58:13.363] [INFO] ClientBinaryManager - Resolving path to Eth client binary ...
[2016-12-06 07:58:13.363] [INFO] ClientBinaryManager - Eth client binary path: /opt/Ethereum Wallet/nodes/eth/linux-x64/eth
[2016-12-06 07:58:13.663] [INFO] ClientBinaryManager - Initializing...
[2016-12-06 07:58:13.664] [INFO] ClientBinaryManager - Resolving platform...
[2016-12-06 07:58:13.664] [INFO] ClientBinaryManager - Calculating possible clients...
[2016-12-06 07:58:13.667] [INFO] ClientBinaryManager - 1 possible clients.
[2016-12-06 07:58:13.667] [INFO] ClientBinaryManager - Verifying status of all 1 possible clients...
[2016-12-06 07:58:13.669] [INFO] ClientBinaryManager - Verify Geth status ...
[2016-12-06 07:58:13.691] [INFO] ClientBinaryManager - Checking for Geth sanity check ...
[2016-12-06 07:58:13.693] [INFO] ClientBinaryManager - Checking sanity for Geth ...
[2016-12-06 07:58:13.764] [INFO] Sockets/node-ipc - Connect to {"path":"/home/imran/.ethereum/privatenet/geth.ipc"}
[2016-12-06 07:58:13.768] [INFO] Sockets/node-ipc - Connected!
[2016-12-06 07:58:13.769] [INFO] NodeSync - Ethereum node connected, re-start sync
[2016-12-06 07:58:13.770] [INFO] NodeSync - Starting sync loop
[2016-12-06 07:58:13.771] [INFO] Sockets/7 - Connect to {"path":"/home/imran/.ethereum/privatenet/geth.ipc"}
[2016-12-06 07:58:13.772] [INFO] main - Connected via IPC to node.
[2016-12-06 07:58:13.801] [INFO] Sockets/7 - Connected!
[2016-12-06 07:58:13.818] [INFO] (ui: splash) - network is privatenet
[2016-12-06 07:58:14.939] [INFO] updateChecker - App is up-to-date.
```

Рис 38 - Запуск кошелька Ethereum с подключением к частной сети по IPC

Сразу после запуска вы увидите интерфейс клиента Mist, который недвусмысленно сообщает о том, что он работает в режиме частной сети (PRIVATE NET) (рис. 12.12).

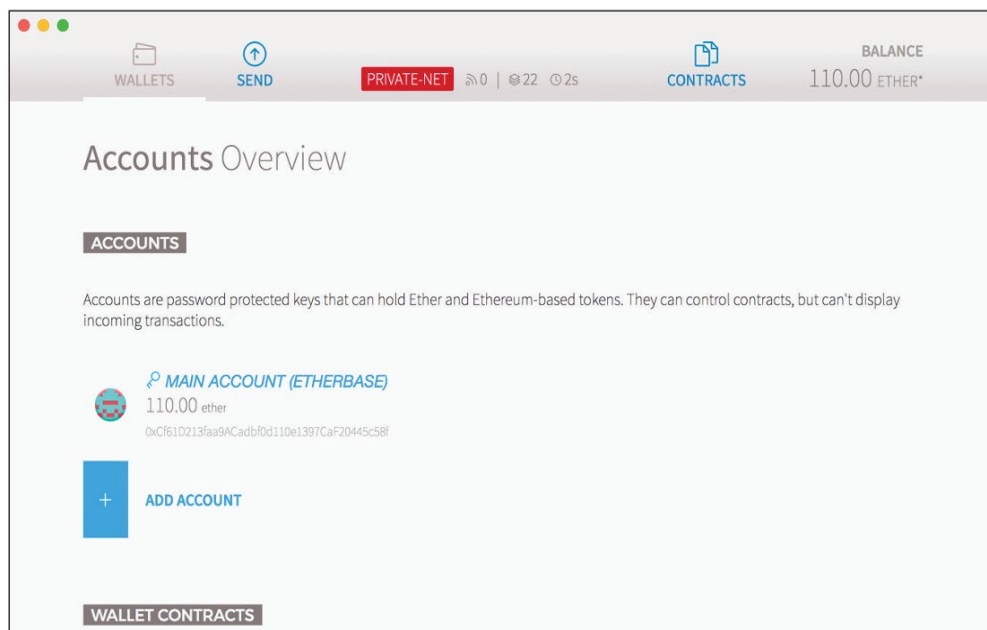


Рис 39 - Mist в частной сети

На рис. 39 можно видеть такие параметры:

WALLETS. Открывает кошельки, если таковые имеются;

SEND. Используется для перевода средств другой учетной записи;

CONTRACTS. Используется для отображения панели контрактов, на которой можно создавать новые и разворачивать уже имеющиеся контракты;

BALANCE. Текущий баланс Ether. Кроме того, вверху показан тип сети, в которой мы находимся. В данном случае это PRIVATE-NET.

Mist умеет работать с сетью через RPC. Это может пригодиться в случае, если клиент Geth запущен на другом узле. При запуске Mist достаточно указать следующий флаг: `--rpc http://127.0.0.1:8545`

Развертывание контрактов с помощью Mist

Развертывание новых контрактов в Mist выполняется крайне просто. Для этого предусмотрен интерфейс, в котором можно писать код на языке Solidity и запускать его в сети.

Ниже показан пример с простым контрактом, который способен выполнять простые арифметические вычисления с входящими параметрами. Но поскольку знакомство с Solidity еще впереди, основное внимание здесь будет уделено процессу развертывания этого контракта и взаимодействию с ним.

Если вы уже знакомы с JavaScript или другим похожим языком, таким как C, этот код покажется вам очевидным.

Пример исходного кода контракта показан ниже:

```
pragma solidity ^0.4.0; A
contract SimpleContract2
{ uint z;
function addition(uint x) public returns (uint y)
```

```

{ z=x+5;
y=z;
}
function difference(uint x) public returns (uint y)
{ z=x-5;
y=z;
}
function division(uint x) public returns (uint y)
{ z=x/5;
y=z; }
function currValue() public view returns (uint)
{ return z; } }

```

Этот код можно просто скопировать в раздел CONTRACTS (Контракты), как показано в левой части снимка экрана; в случае отсутствия синтаксических ошибок в правой части появится раскрывающийся список под названием SELECT CONTRACT TO DEPLOY (Выберите контракт для развертывания). Просто выберите свой контракт и нажмите кнопку Deploy (Развернуть) внизу окна (рис. 40).

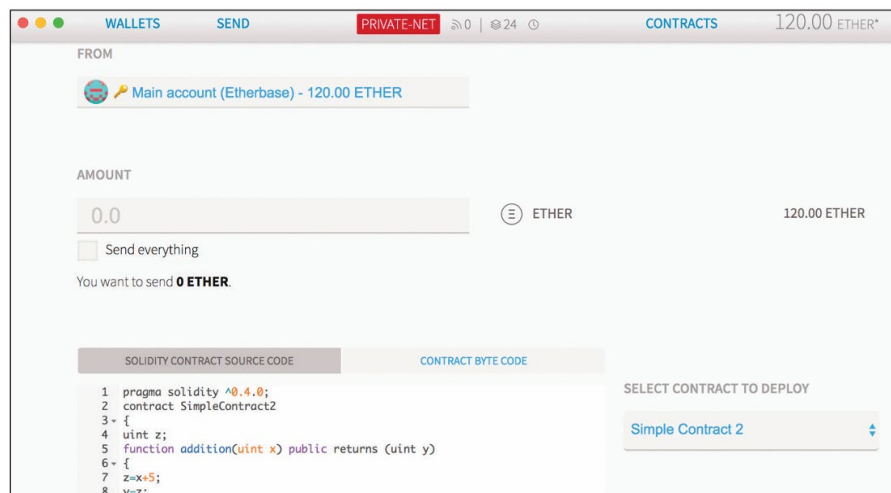


Рис 40 - Развертывание контрактов в браузере Mist

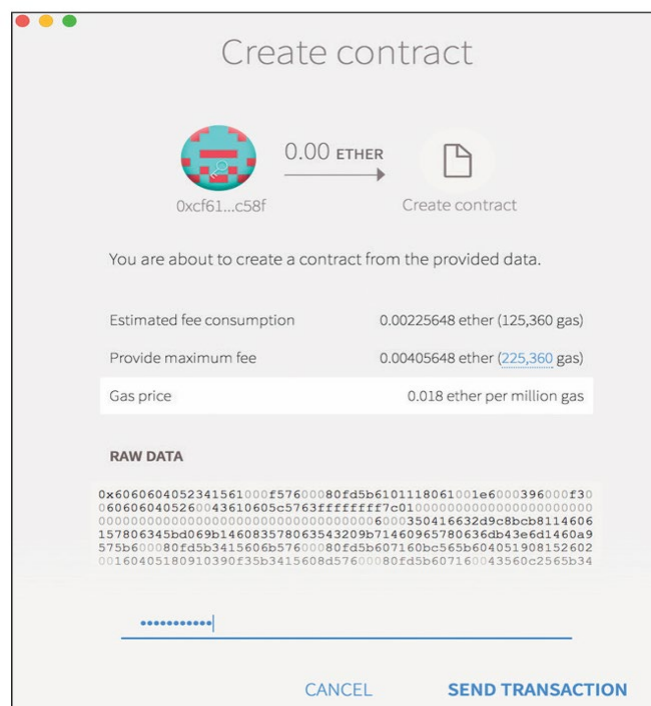


Рис 41 - Создание контракта с помощью Mist

Mist запросит у вас пароль к учетной записи и выведет окно, показанное на рис. 41. Введите пароль и нажмите кнопку **SEND TRANSACTION** (Отправить транзакцию), чтобы развернуть контракт. После успешного развертывания и майнинга контракт появится в списке транзакций Mist, как показано на рис. 42.

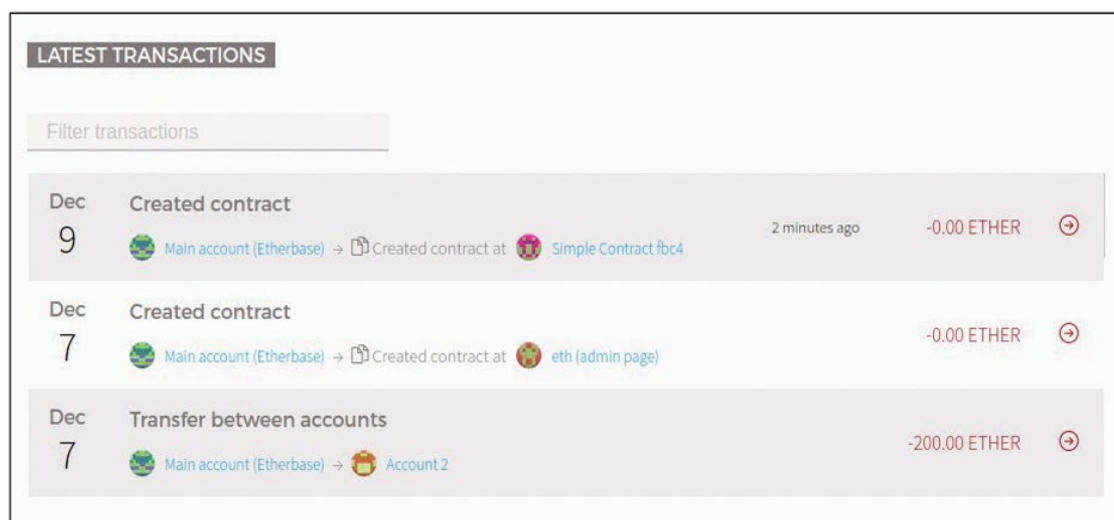


Рис 42 - список транзакций Mist

Список транзакций после создания контракта в Mist

Как только контракт станет доступным, с ним можно будет взаимодействовать посредством функций, которые он предоставляет. Это можно видеть на рис. 12.16: в правой части окна находится раскрывающийся список со всеми функциями, доступными из контракта.

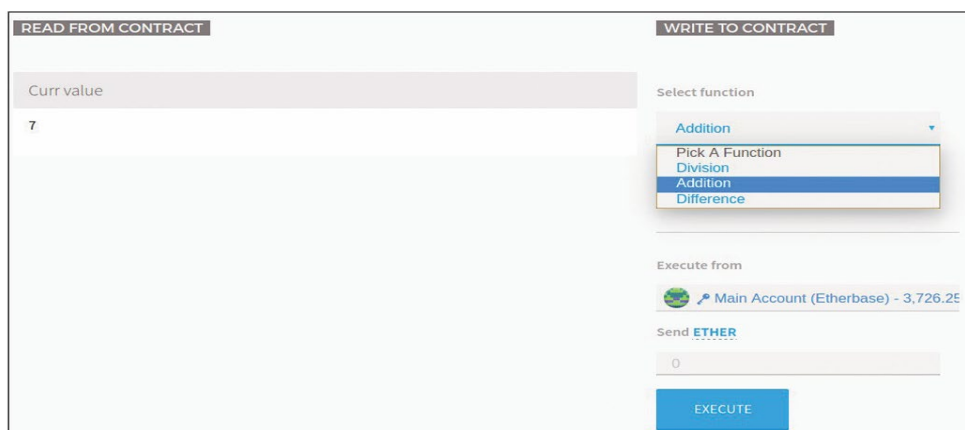


Рис 43 - Взаимодействие с контрактом с помощью параметров чтения и записи в Mist

Рисунок 43 состоит из разделов READ FROM CONTRACT (Чтение из контракта) и WRITE TO CONTRACT (Запись в контракт). Кроме того, справа выводятся функции, предоставляемые контрактом. Выбрав нужную функцию, введите подходящее значение, укажите учетную запись в разделе Execute from (Выполнить из) и нажмите кнопку EXECUTE (Выполнить). В результате запустится транзакция, которая вызовет выбранную функцию контракта. Этот процесс показан на рис. 44

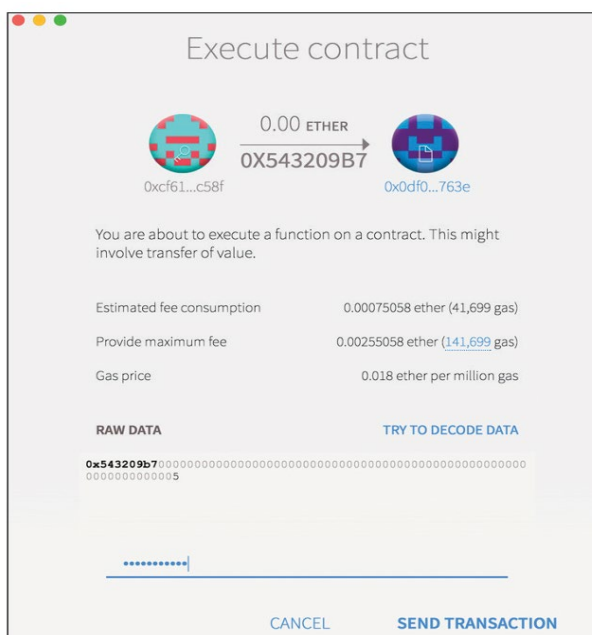


Рис 44 - Выполнение контракта в Mist

Чтобы отправить транзакцию контракту, введите пароль учетной записи и нажмите кнопку SEND TRANSACTION (Отправить транзакцию), как это по казано выше. Обозреватель блоков для частных/локальных сетей Ethereum Обозреватель блоков Ethereum – это полезный инструмент, который помогает просматривать локальный частный блокчейн.

На странице GitHub github.com/etherparty/explorer доступен бесплатный обозреватель блоков с открытым исходным кодом под названием EthExplorer. Мы будем использовать его в наших примерах для визуализации блоков и транзакций.

Установить EthExplorer можно следующим образом:

1. В Linux Ubuntu или macOS нужно запустить следующую команду, кото рая загрузит EthExplorer: `$ git clone https://github.com/etherparty/explorer` Вы должны увидеть примерно

такой вывод: Cloning into 'explorer'... remote: Counting objects: 269, done. remote: Total 269 (delta 0), reused 0 (delta 0), pack-reused 269 Receiving objects: 100% (269/269), 59.41 KiB | 134.00 KiB/s, done. Resolving deltas: 100% (139/139), done.

2. Далее следует перейти в загруженную директорию и выполнить стартовый скрипт: `$ cd explorer/ $ npm start`

Если на компьютере, который вы используете, не установлен пакет Node.js, вы можете загрузить его на официальном веб-сайте, nodejs.org/en/. Там же находятся инструкции по установке.

После завершения установки (может занять почти 5 минут) вы увидите вывод, сигнализирующий о запуске HTTP-сервера для EthExplorer (рис. 45):



```
node-uuid@1.4.8
oauth-sign@0.8.2
qs@3.1.0
stringstream@0.0.6
tough-cookie@2.3.3
punycode@1.4.1
tunnel-agent@0.4.3
saucelabs@1.0.1
https-proxy-agent@1.0.0
agent-base@2.1.1
semver@5.0.3
extend@3.0.1
selenium-webdriver@2.47.0
tmp@0.0.24
ws@0.8.1
bufferutil@1.2.1
bindings@1.2.1
options@0.0.4
ultron@1.0.2
utf-8-validate@1.2.2
nan@2.4.0
xml2js@0.4.4
sax@0.6.4
xmlbuilder@9.0.4
source-map-support@0.2.10
source-map@0.1.32
shelljs@0.2.4

> EthereumExplorer@0.1.0 start /Users/drequinox/explorer
> http-server ./app -a localhost -p 8000 -c-1

Starting up http-server, serving ./app on port: 8000
Hit CTRL-C to stop the server
```

Рис 45 - HTTP-сервер EthExplorer

3. Как только сервер начнет работать, вы должны запустить клиента geth, используя следующую команду: `geth --datadir .ethereum/privatenet/ --networkid 786 --rpc--rpccorsdomain 'http://localhost:8000'` Как вариант эту команду можно немного изменить: `geth --datadir .ethereum/privatenet/ --networkid 786 --rpc`

4. После успешного запуска geth откройте localhost на порту 8000, чтобы получить доступ к EthExplorer (рис. 46):

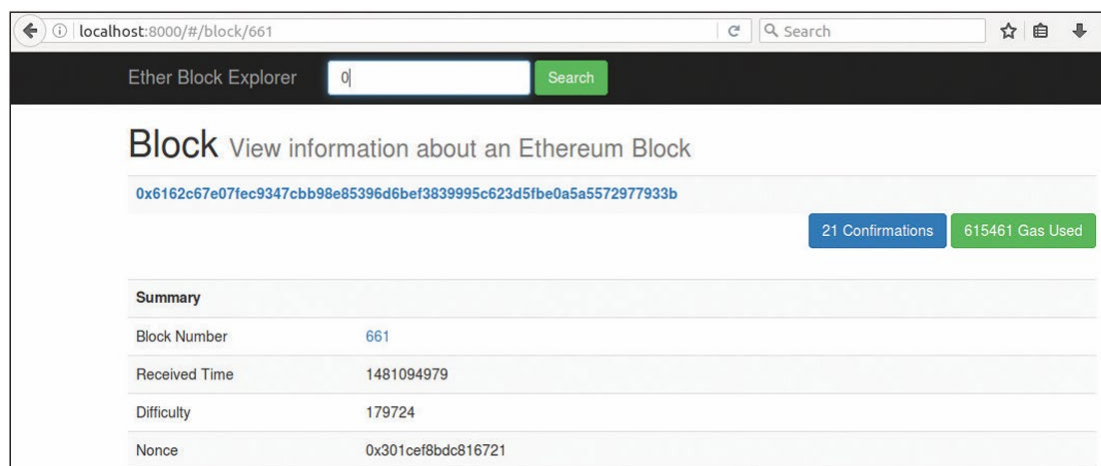


Рис 46 - Обзорщик блоков

В качестве альтернативы можно использовать любой другой веб-сервер. Например, вы можете воспользоваться простым веб-сервером, встроенным в Python:

```
$ python -m SimpleHTTPServer 7777
```

Serving HTTP on 0.0.0.0 port 7777 ...

Клиент geth должен быть запущен с подходящими параметрами, иначе может возникнуть ошибка, похожая на показанную на рис. 47.



Рис 47 - Сообщение об ошибке

Перезапустите geth с флагом `--rpccorsdomain`:

```
./geth --datadir ~/etherprivate/ --networkid 786 --rpc --rpcapi 'web3,eth,net,debug,personal'
```

Символ * означает, что подключаться можно с любого IP-адреса. Вместо этого можно указать локальный адрес своего компьютера.